

🏠 Home ▾

👤 Identities ▾

⚙️ Customization ▾

🗺️ Journeys ▾

🛡️ Consent ▾

🛡️ Security ▾

🔗 Integrations ^

Applications

Identity providers

Email gateway

Self-service

Set up and configure the Self-service application. To learn more, [read the documentation](#) 📄.

Attributes Security

Profile attributes

Profile attributes originate from the Identity Store and appear on the Personal Information page, where users can view and edit them using a standard form interface.

🔍 Search items

+ Select

Display name	Type	Is visible	Is required	Mutability	Status
Salutation	string	☒	☐	Read write	Active
First name	string	☒	☒	Read write	

Self-service Portal

A cross-device self-service experience,
shaped by administrator settings

OUTCOMES

The platform now has a self-service experience aligned with its capabilities and supports complex configuration without compromising usability.

GOAL

The capability needed to be configurable, brandable, and balance consumer-grade usability with enterprise requirements.

CLIENT AND SCOPE

The internal client for this project was Thales’ Identity and Access Management (IAM) organization. The engagement intermittently across 11 months in 2025.

ROLE

I led the design for the Self-Service experience and two upstream application areas that it relied on.

How might we design a better way for admins to configure a self-service experience?

In a digital world, Organizations need help with identity and access management

Organizations today function in a world where identity and access to applications and information has become essential to safely operating.

In recent years this has only compounded, especially for large enterprises. Software and services have become increasingly cloud-based, privacy and regulatory requirements have expanded, employees and customers expect access from anywhere, and sensitive data is frequently targeted by bad actors.

OneWelcome supports end-to-end identity and access for customers and employees

The case is stronger than ever for companies to use [Identity and Access Management \(IAM\)](#) platforms like Thales' OneWelcome Identity Platform to offload the growing complexity of identity and access management to specialized providers.

OneWelcome supports the full identity and access management journey for customers and employees, including onboarding users, signing in, recovering account access, and managing account information, security, consent, and privacy settings.

It is built to support both enterprise and consumer markets, making it essential for the platform to deliver a consumer-grade experience where secure access is as frictionless as possible.

At the same time, organizations needed end-user experiences to adapt to their unique security policies and regulatory requirements. OneWelcome addressed this through a highly configurable platform model, enabling IAM administrators to shape how identity and access experiences behaved based on their organization's security needs.

IAM platforms perform many functions and Self-service is a foundational element

The concept of self-service in identity and access management has existed for decades, enabling users to manage credentials, access, and profile data while reducing support overhead. However, out-of-the-box self-service experiences are often limited in functionality and lag in usability.

Organizations either accept these limitations or consume self-service capabilities via APIs to build custom experiences tailored to their branding and functional requirements.

This approach provides greater flexibility and control. However, it shifts significant

development and maintenance burden onto customers. It also increases long-term platform complexity for vendors, which must support stable, evolving interfaces for custom implementations.

Our vision was a configurable, branded self-service reducing custom builds

We believed there was a better way. Self-service should be a first-class IAM capability, not an afterthought.

The vision for self-service was to meet organizations' IAM needs while providing end users with comprehensive self-service, branded experiences, and consumer-grade usability across devices –out of the box.

A key aspect of this vision was extensive configurability for admins, allowing them to tailor the experience to specific security and policy requirements.

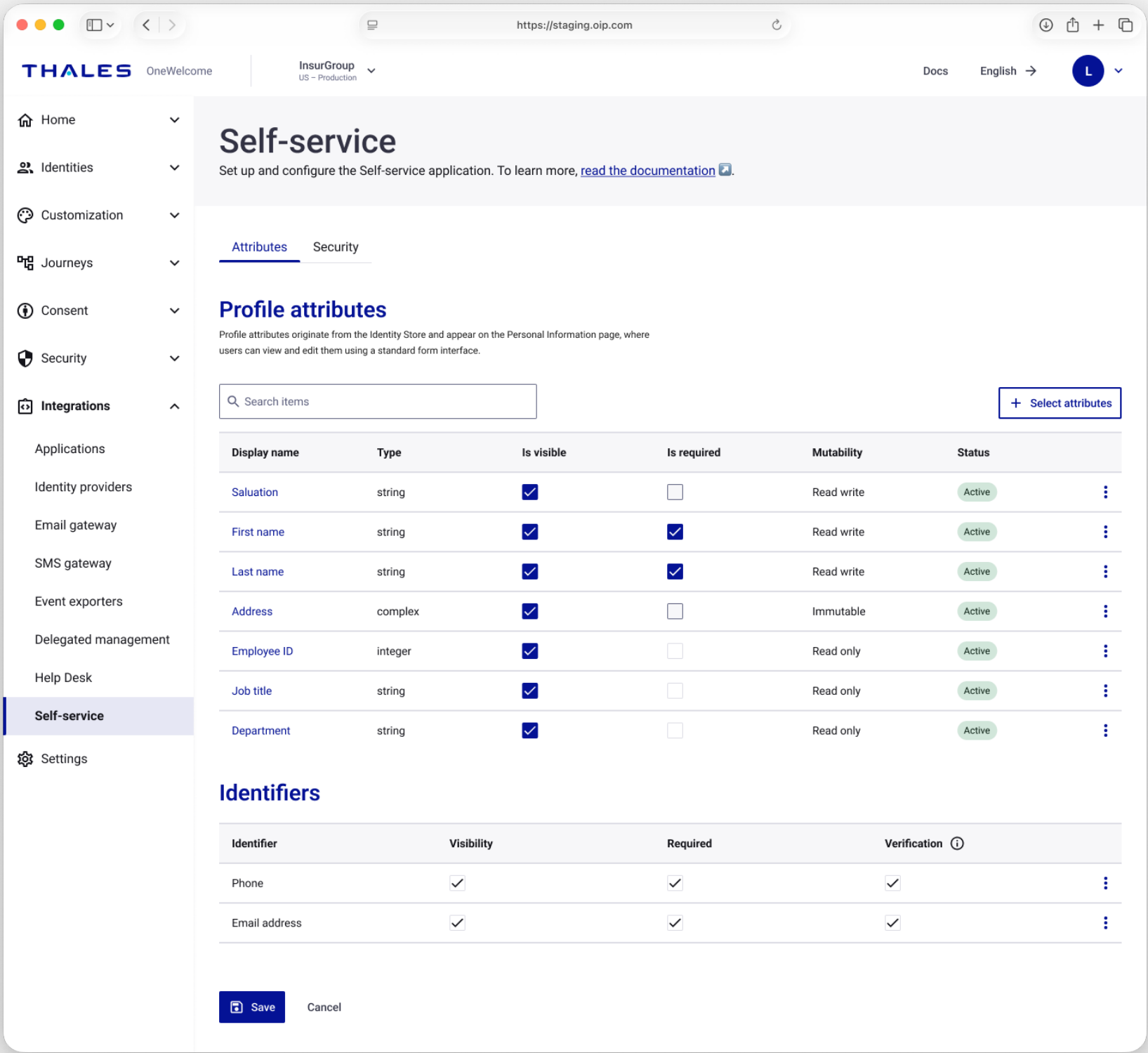
Together, these capabilities positioned self-service as a compelling platform offering, reducing the need for customers to build fully custom experiences from scratch.

Admins can configure experiences to meet their organization's security needs

Organizations need self-service experiences that adapt to their unique security policies, operational requirements, and regulatory constraints.

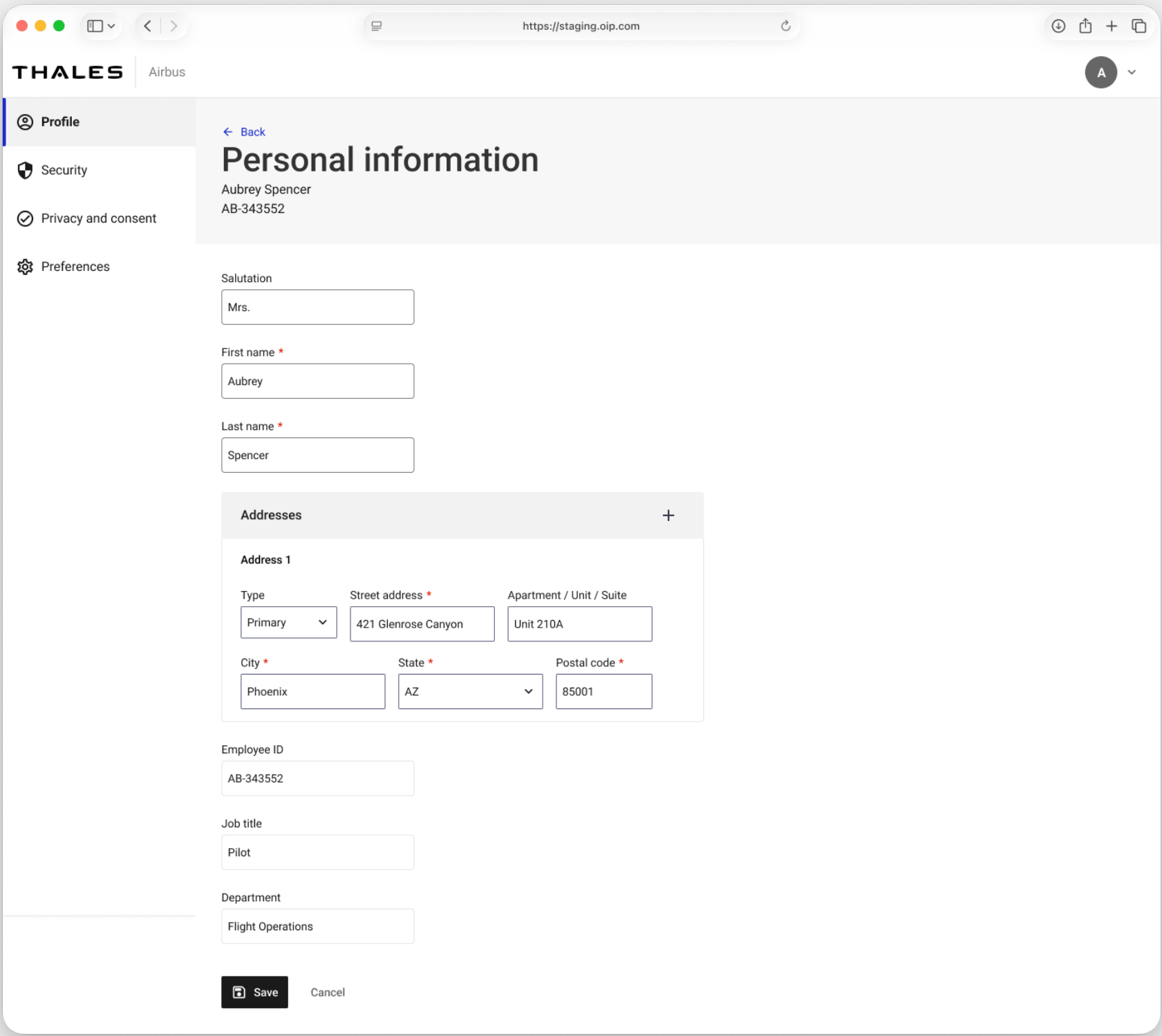
I designed Self-service as a composable system that let administrators granularly control the end-user experience across page sections, profile attributes, and authentication methods.

SOLUTION



End user experience

End users, such as employees and customers, experience the results of an admin's profile configuration choices.



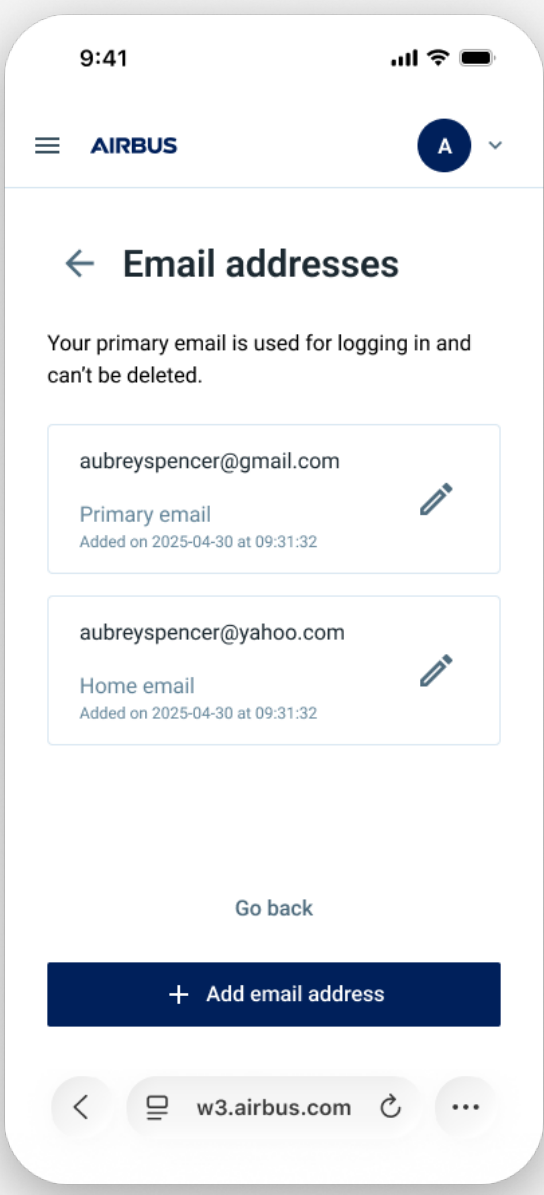
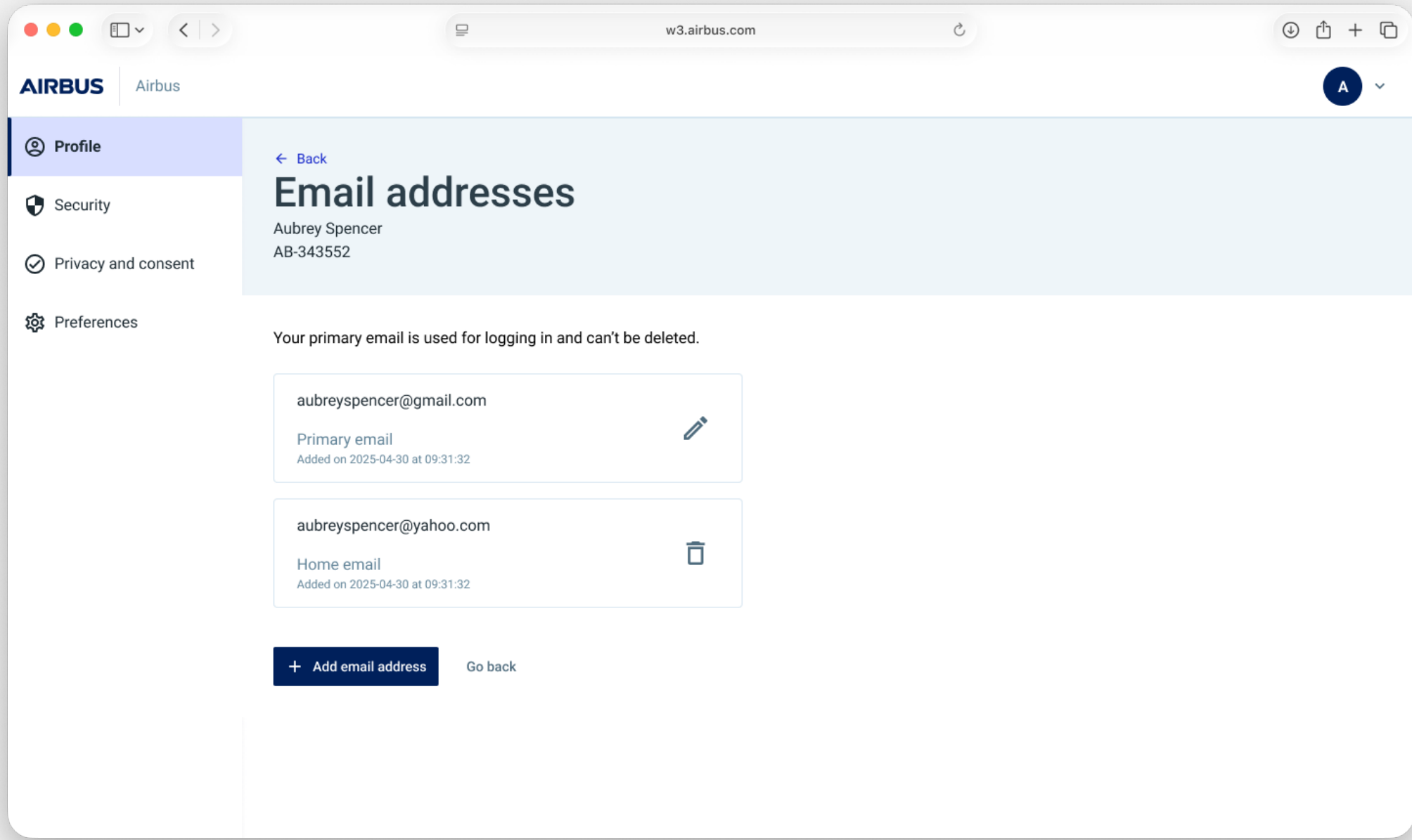
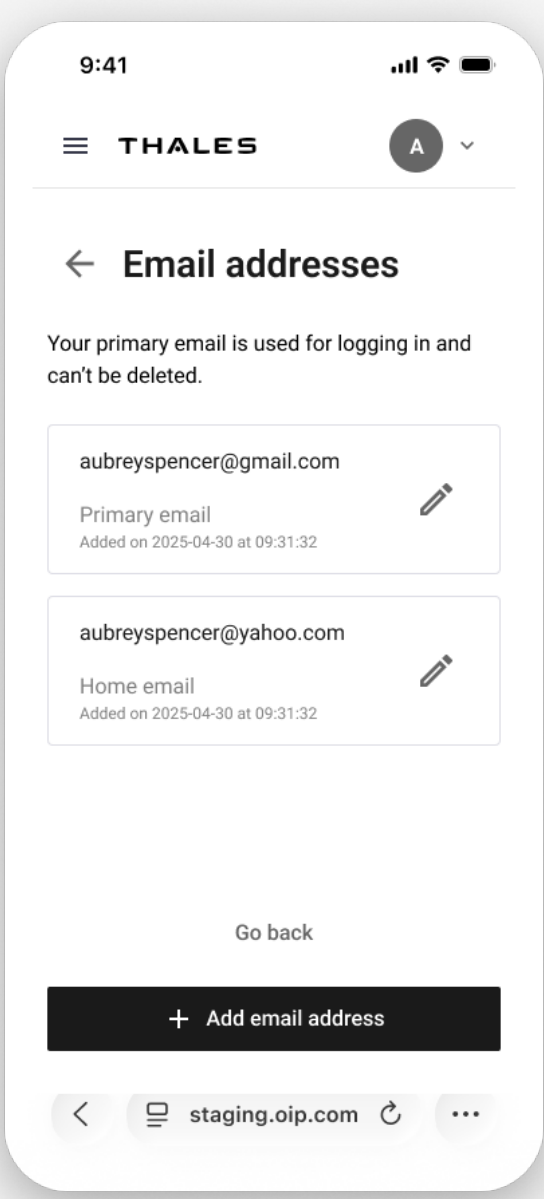
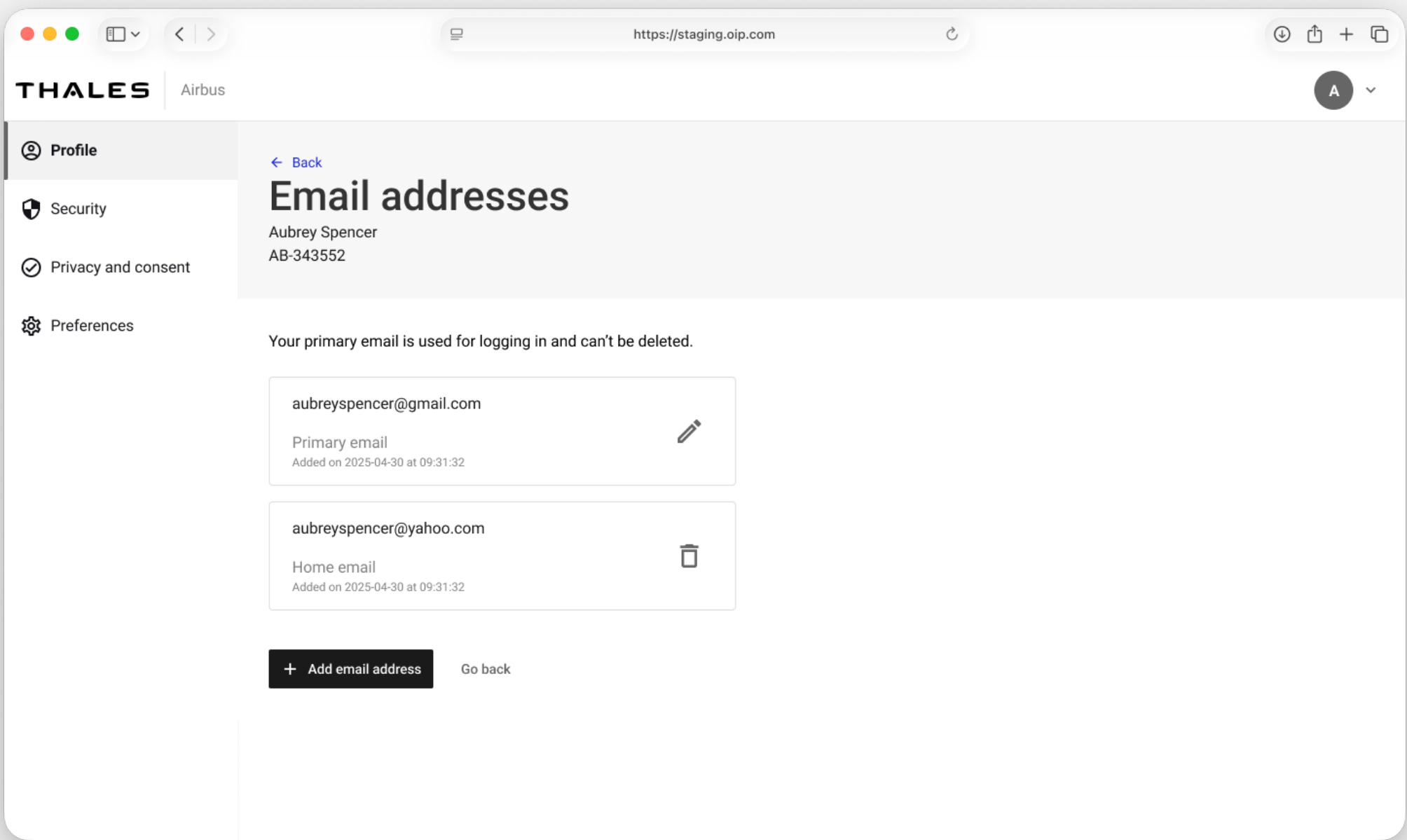
Admin experience

Admins define which attributes appear on a user's profile and how users can interact with them.

Admins customize and end users receive a branded cross-device experience

Self-Service needed to support both enterprise and consumer organizations where end users receive branded, cross-device experience.

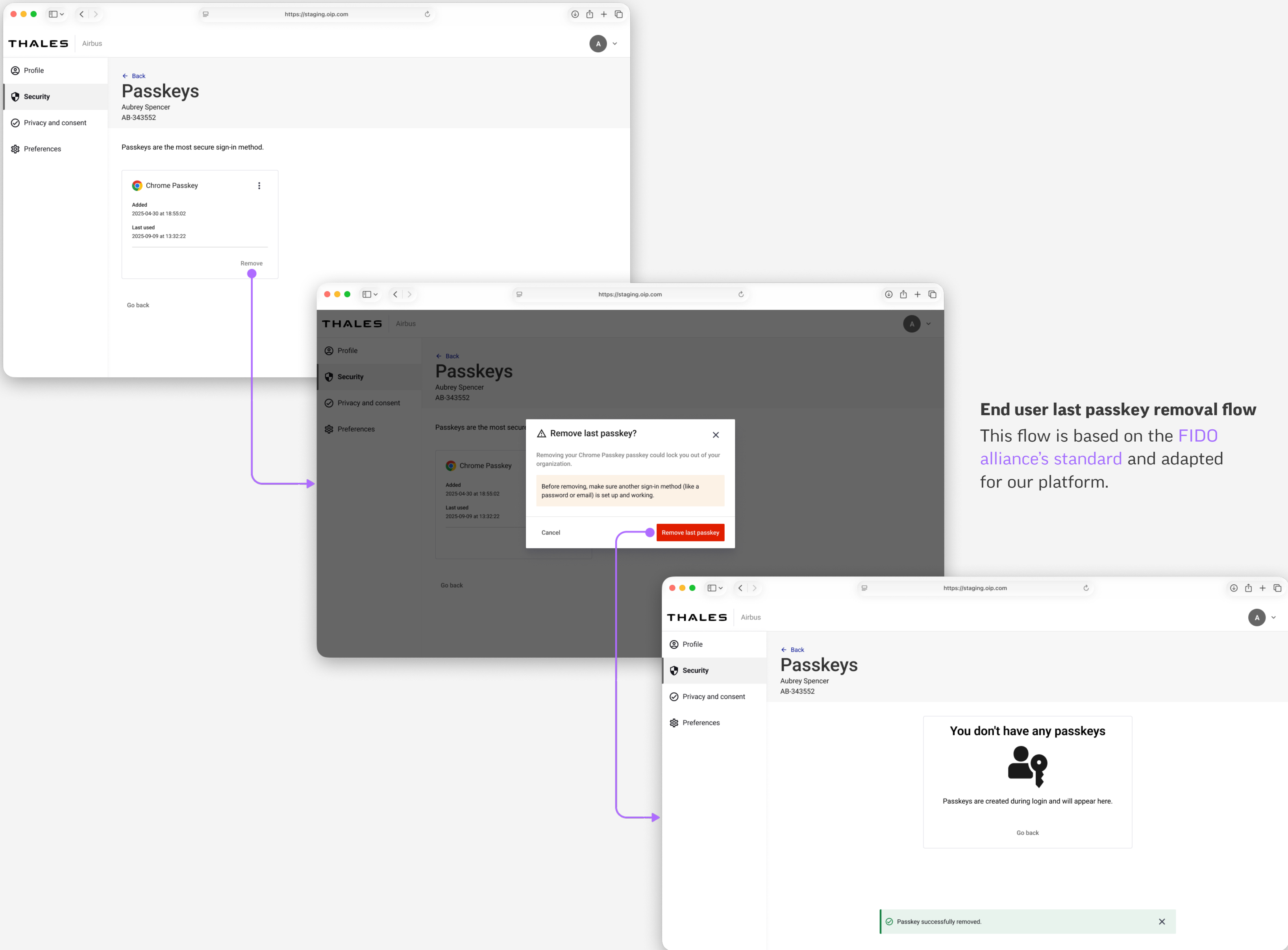
SOLUTION



Minimal branding with big impact
Token-based theming provides impactful branding while keeping admin configuration minimal.

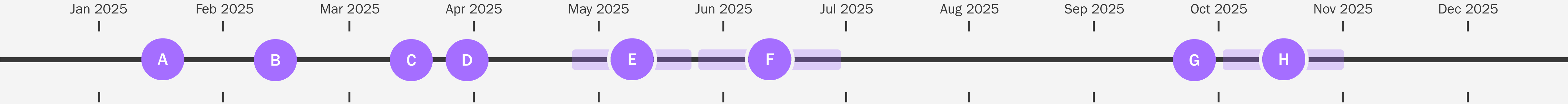
Our platformed balanced the competing needs of rigorous security standards and intuitive user experiences.

Self-service prioritized industry standard authentication and verification workflows, giving users a familiar mental model that extended beyond our product without compromising security.



Self-service Portal timeline
11 months, 2025

PROCESS



(A) Self-service kicks off
The project began by prioritizing consumer organizations over enterprise organizations and defining the core design principles.

(B) Shifting toward enterprise organizations
A vision playback revealed stakeholder misalignment, resulting in a shift away from consumer organizations.

(C) Defining branding and theming
I designed and aligned the UI customization vision within Self-Service.

(D) Uncovering SCIM 2.0 constraints
I designed and presented prototypes for end-user and admin experiences, uncovering SCIM 2.0 constraints.

(E) Implementation reveals constraints
Early into supporting implementation, legacy platform constraints altered scope for Self-services first release.

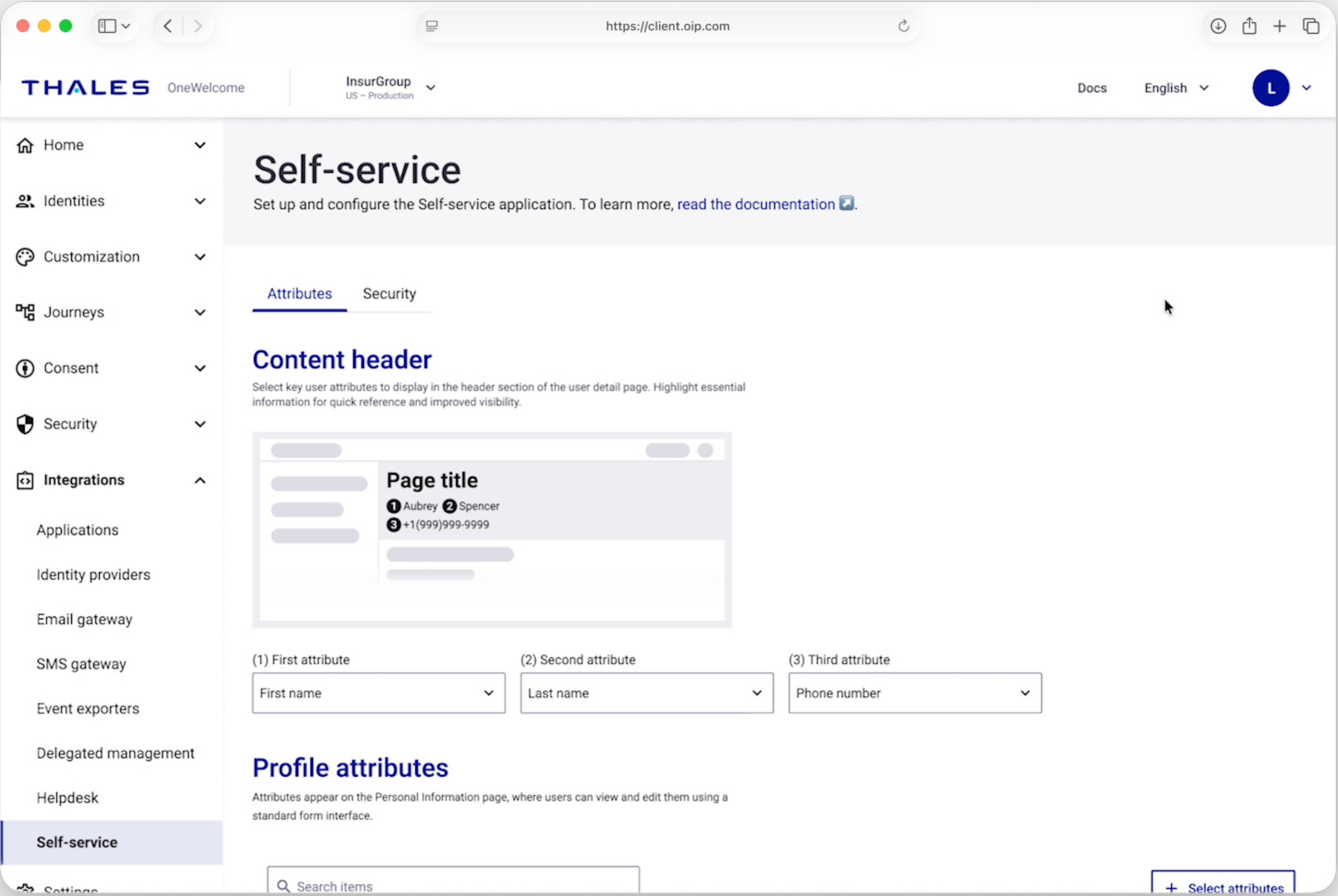
(F) Working through SCIM 2.0 constraints
I spent time collaborating with a designer from Help Desk to work through numerous challenges with SCIM 2.0.

(G) Converged on a design for Applications
Applications was long riddled with misalignment, but my renewed focus brought the team into a shared direction.

(H) Designing authentication management
I designed and supported implementation of end-user and admin experiences for managing passkeys and linked accounts.

The most complex challenge was to design a configurable user profile

This was complex because flexible profile configuration had to comply with SCIM 2.0 requirements while also integrating with legacy platform components.



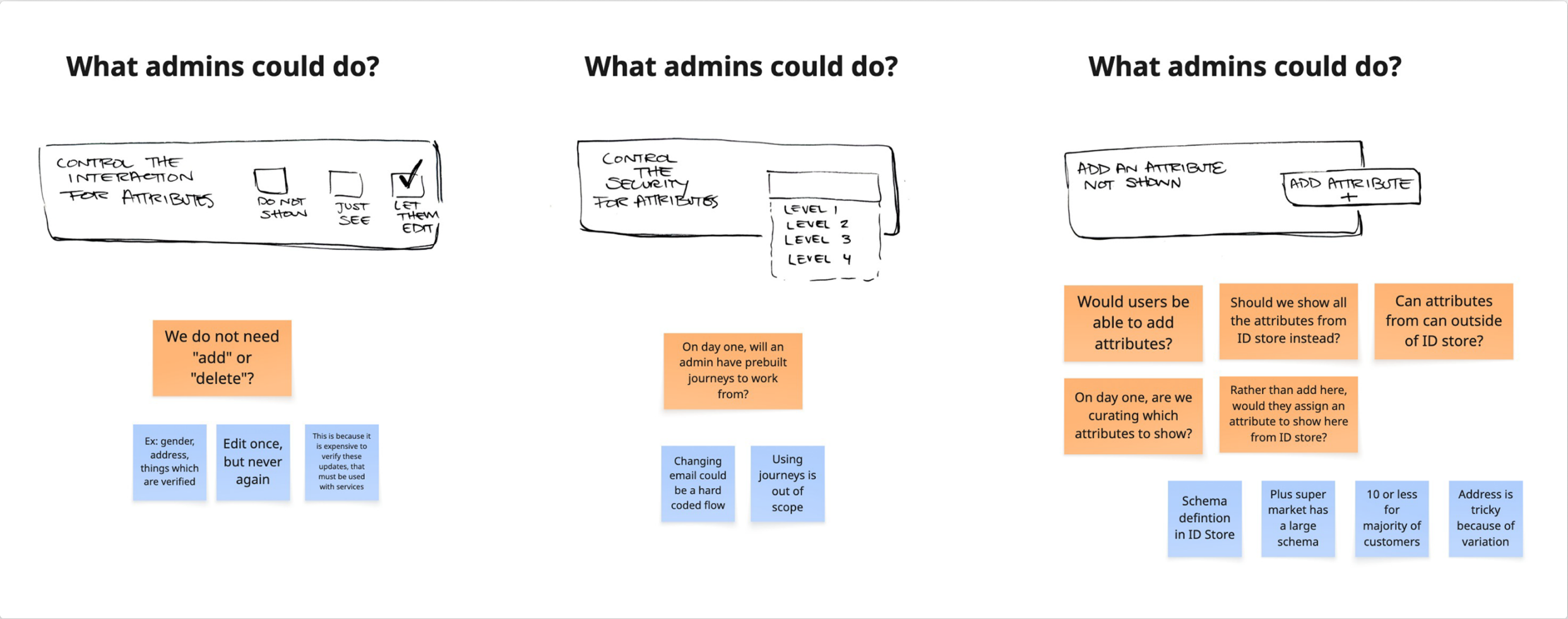
Profile complexity was driven by SCIM-based User Store constraints

During discovery, the initial requirements focused solely on end-users quickly evolved. I learned that the profile wasn’t fixed, depended on another core platform module called the User Store.

The User Store manages customer identities and their associated attributes and identifiers (A). Attributes are pieces of information associated with a user, such as their first name. Identifiers are also attributes and identify a person logging in, such as an email address.

Its schema complied with SCIM 2.0 standards, which introduced architectural constraints and had far-reaching impacts on the design of Self-service’s user profile.

PROCESS



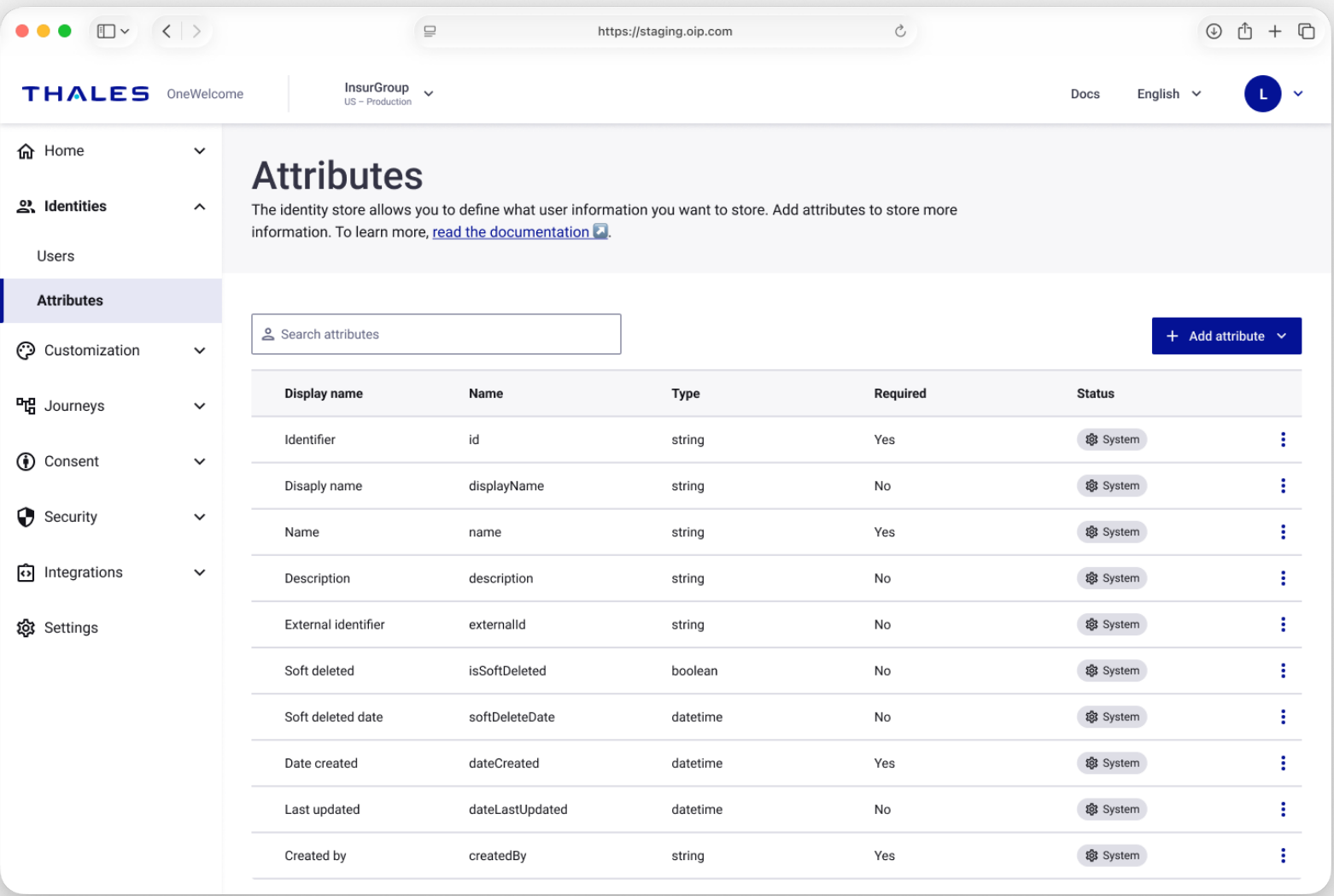
Pressure-testing project scope

These sketches showed functionality without laying out screens and was used to quickly move Self-service out of abstract requirements.

A

The User Store powers the platform

Admins define attributes to power profile data, policies, and integrations across the platform.

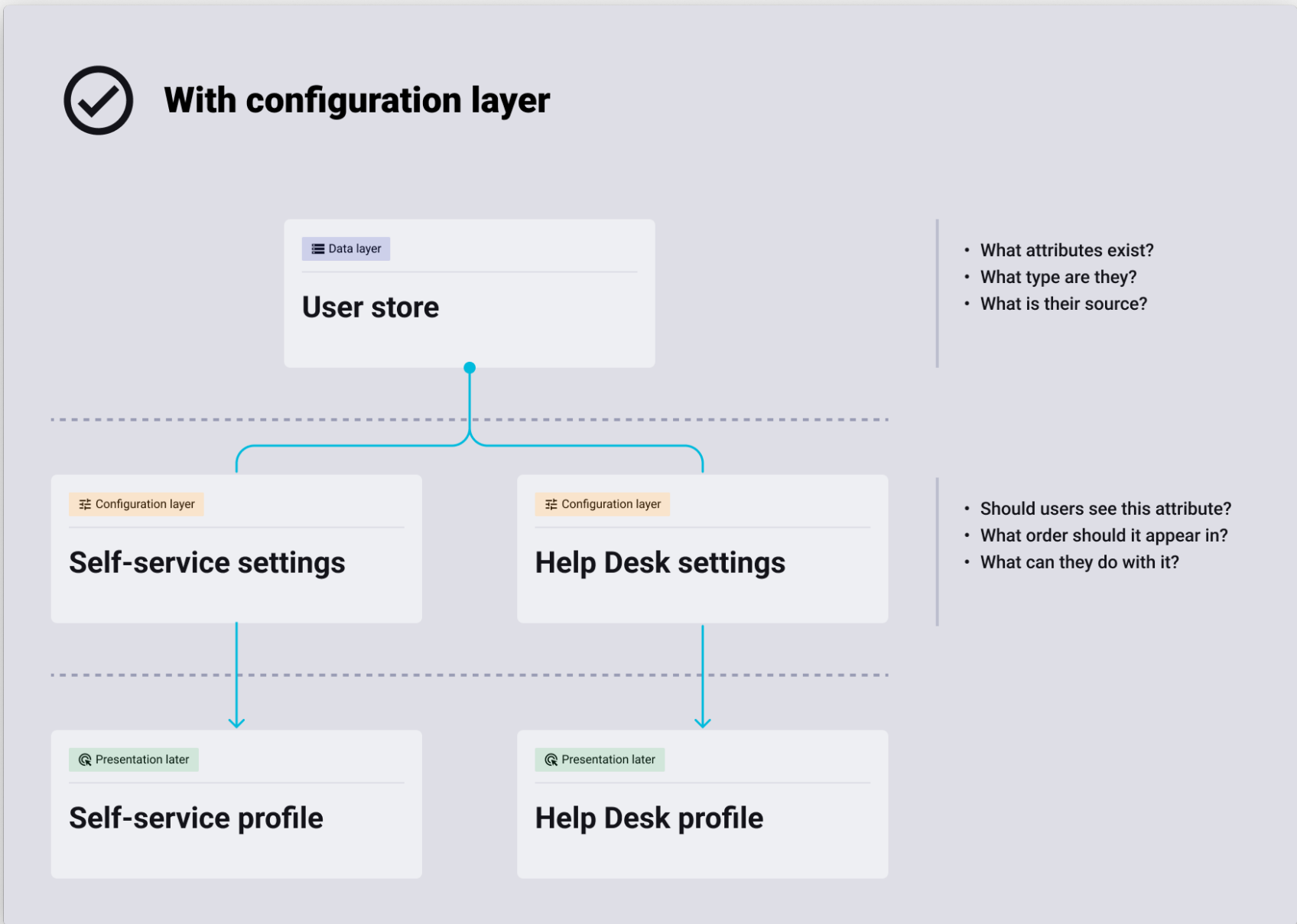


I separated attribute definition from profile configuration to reduce complexity

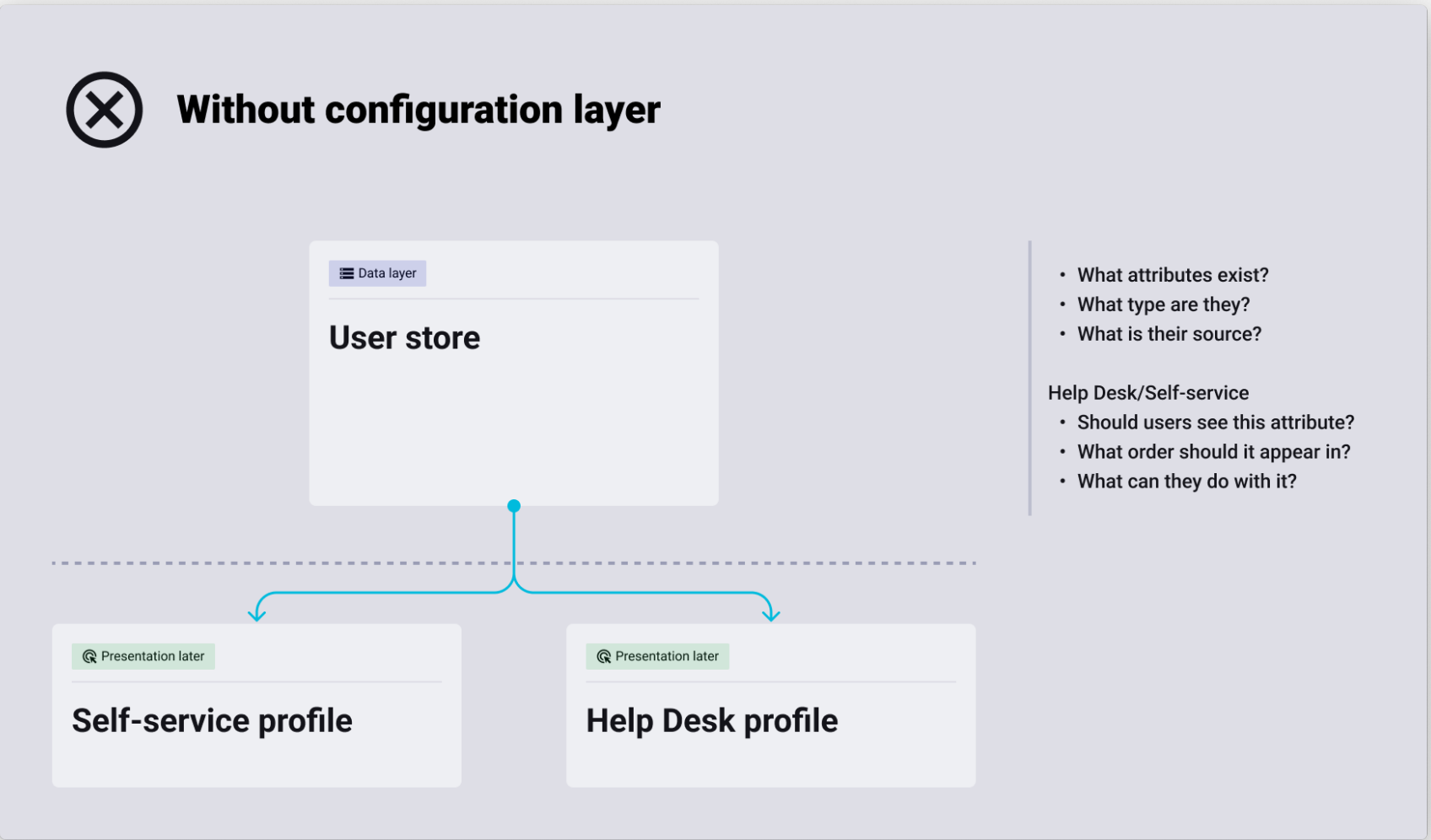
A formative decision I made was to require admins to configure the user profile outside the User Store, where profile attributes originate (A).

Research supported that defining attributes in the User Store and configuring a profile were independent tasks, each with its own goal and sometimes performed by different admins. Also a factor, profile configuration was split between Self-service and Help Desk experiences.

Forcing these mental models into the User Store blurred the boundaries between identity architecture and profile management. The interaction cost would be high, as admins would need to navigate a growing matrix of concerns, switching between defining data and configuring how it was used (B).



A



B

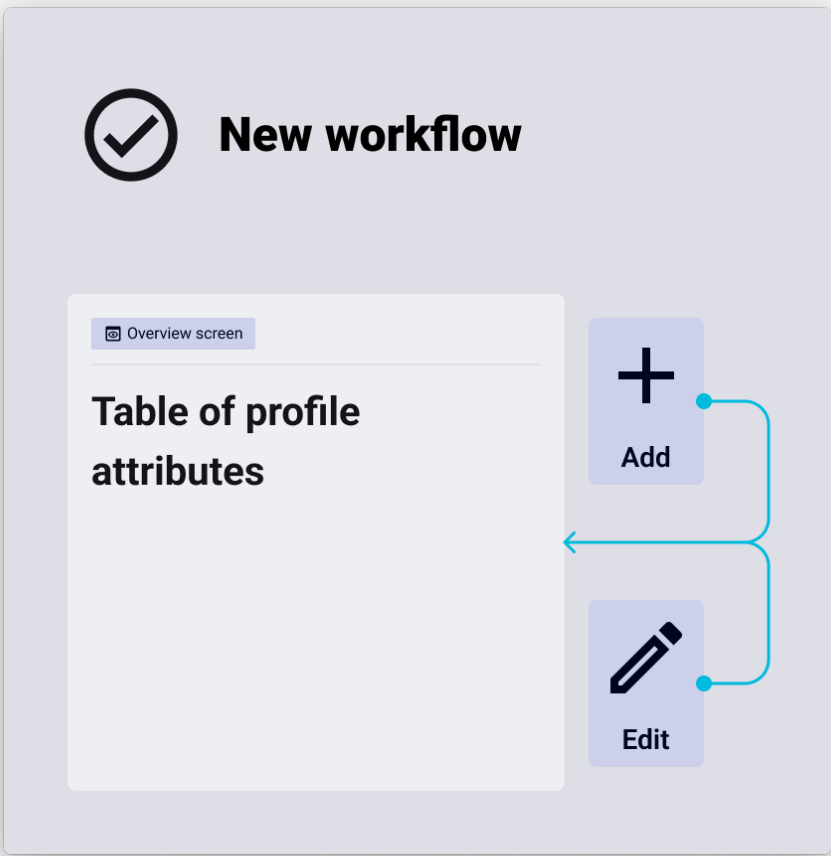
I traded consistency to simplify how admins selected and edited attributes

The admin’s first step in building a user profile is curating a subset of User Store attributes, since not all are relevant to end users (A).

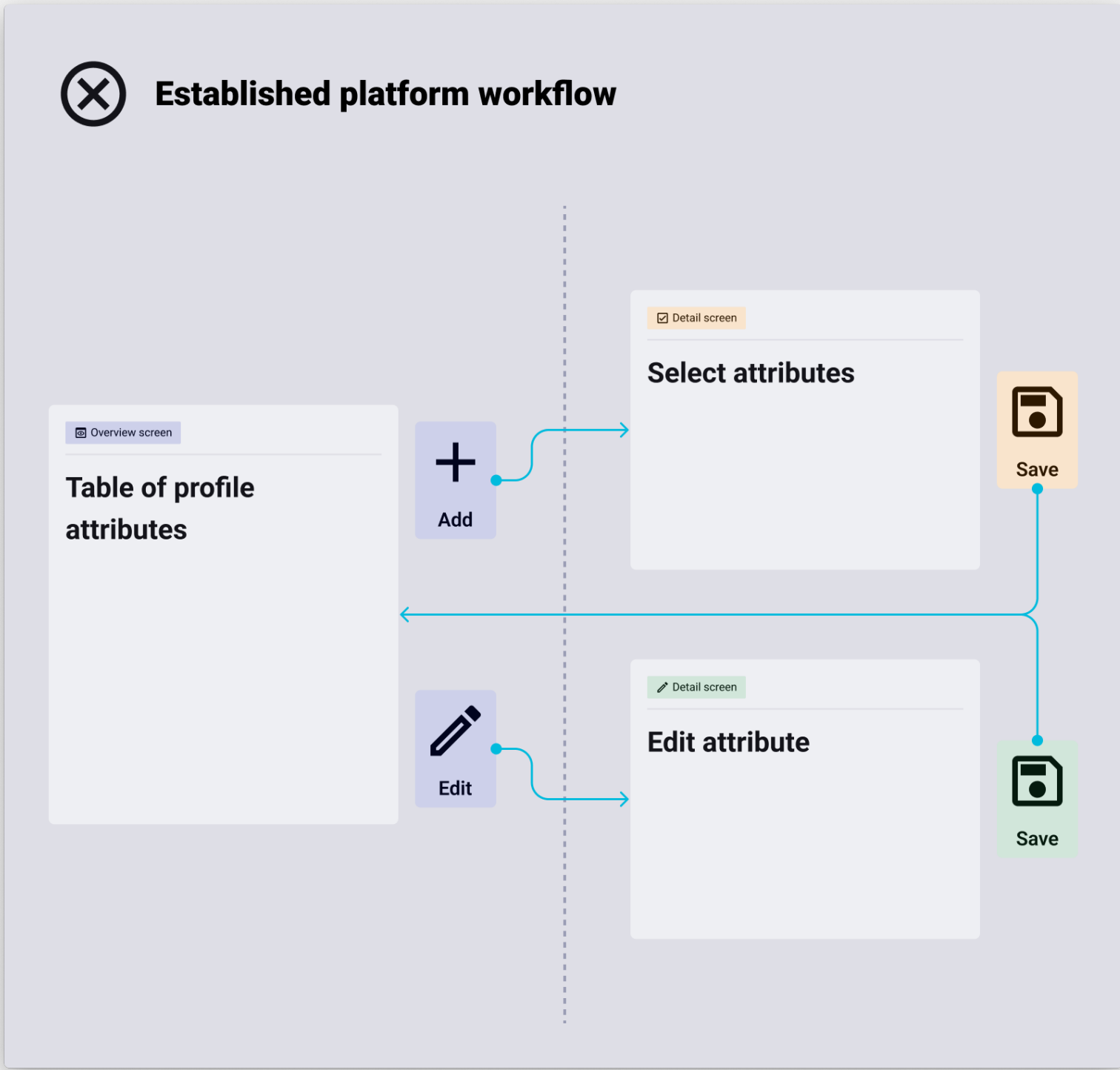
To perform this task with our platform’s standard workflow, admins would navigate from a data table to dedicated edit views and back (B). While using this pattern reinforces product consistency, I believed it introduced too much navigation and interaction for profile configuration.

I introduced a new workflow where admins selected attributes, edited properties, and controlled ordering directly in a table, enabling tight feedback loops while maintaining visibility of the full profile (C).

C



B



Attributes operate in two contexts

Identity system attributes exist only for integrations, authorization, or security policies.

User profile attributes	Identity system attributes
First Name	Risk Score
Last Name	Failed Login History
Email Address	Device Trust ID
Phone Number	Audit Event ID

A

I defined inline editing to meet immediate needs while enabling it to scale over time

A core part of configuring a user profile was controlling attribute visibility, validation rules, and permissions.

These tasks were intended to be supported by the inline editing workflow introduced earlier. However, the current data table component did not support this capability.

As the design system lead, I defined an inline editing interaction model and applied it first to checkbox and select components, the only ones needed for the admin experience.

This approach balanced development effort and established a scalable model for future extension.

Defining how inline editing works

I reviewed inline editing patterns in enterprise data grids and defined a model aligned with usability best practices and our platform's needs.

Interaction model: Direct Interaction Editing

Inline editable tables follow a Direct Interaction Editing model. This approach prioritizes minimal visual noise in the at-rest state, progressive discoverability on hover, and immediate interaction on click.

This interaction model supports:

- Speed: Fewer clicks to accomplish common tasks.
- Clarity: A clean default state with interactions revealed progressively.
- Consistency: Users learn the model once and apply it across all fields.

1. Display state

- Cells appear static and non-editable.
- No visible affordances like input fields, chevrons, or placeholder text.
- The focus is on a clean, readable data presentation.

This reduces visual clutter and makes the table easier to scan.

2. Hover state

- The cursor changes to reflect the cell's interactive nature (e.g., pointer or text cursor).
- Placeholder text may become visible where relevant.
- Optional: In the case of Select components, a chevron may appear on hover to suggest dropdown behavior.

This signals interactivity only when the user engages with the row or cell, balancing discoverability with simplicity.

3. Click or tap interaction

- A single click is sufficient to interact directly with the input element.
- The cell transitions into an inline editing mode with passthrough behavior, meaning:
 - Clicking a checkbox immediately toggles it.
 - Clicking a cell with a Select opens the dropdown.
 - Clicking into a text field places the cursor inside the input for typing.
- There is no need for an intermediate activation step (i.e., no click-to-edit then click-to-interact sequence).

Improves efficiency for power users and reduces friction in common workflows.

4. Edit mode behavior

- Edit mode is transient and scoped to the active cell.
- The component used (e.g., Select, Checkbox, Text Input) behaves as it normally would outside the table.
- Saving changes still must be done through an explicit save at the bottom of the form or overview page.

Handling tap behavior without hover

Touch devices do not support hover, which means some interactive cues—normally revealed on hover—won't be visible. We accept this limitation, recognizing that our product's primary use case is desktop, not touch.

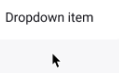
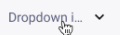
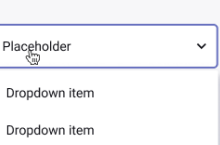
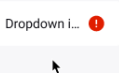
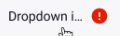
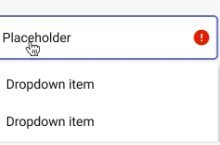
Touch interfaces are exploratory by nature.

- Mobile users are already used to tapping into things and seeing modals, dropdowns, or pickers appear.

There's little cost to tapping.

- If the Select opens a dropdown or a bottom sheet, users can back out just as quickly. It's low risk.

Cursor navigation		
Checkbox		
Display	Hover	Checked
		
• The cursor style does not change when hovering over the cell.	• When hovering over the checkbox, the cursor style changes to a pointer.	• Users can select the checkbox with a single click; it behaves like a standard checkbox.

Select		
Display	Hover	Expanded
		
Before any interaction, the Editable Select Cell is shown in its default display state.	• When the user hovers over the cell, the cursor changes to a pointer, the cell's background color updates, and a chevron icon appears.	• Users can activate the Select dropdown with a single click. It opens above the cell in a consistent, standard size that does not depend on the cell's dimensions.
Display with error	Hover with error	Expanded with error
		
When the Select component has an error, we display an error icon.	• On hover, the cursor changes to a pointer and the cell's background color updates. If the cell has an error, the error icon remains visible and shows a tooltip on hover.	• The error icon remains visible while the dropdown menu is open, and only disappears once the error is resolved.

Keyboard navigation				
Pressing Esc at any time will cancel the action and clear focus from all elements.				
Checkbox				
	☒	☒		
• Tab moves focus directly to the checkbox.	• Spacebar toggles the checkbox.	• Tab again moves focus to the next element.		
⬅️	Space	⬅️		
Select				
Dropdown item	Placeholder Option 1 Option 2	Placeholder Option 1 Option 2	Option 2	Option 2
• Tab moves focus to the cell.	• Spacebar opens the Select component.	• Arrows navigate the Dropdown menu. Escape to cancel.	• Enter to select	• Tab again moves focus to the next element.
⬅️	Space	⬆️⬆️⬅️	Enter	⬅️

Supporting keyboards and cursors

Inline editing needed to be accessibility, so it needed to support both cursor and keyboard interactions.

The solution for ordering attributes evolved with scope change

Ordering attributes was key to shaping a profile. This became complex quickly, as the roadmap required supporting ordering across multiple pages, not just a single page.

I sketched out patterns to handle this workflow with existing components and accessibility in mind (A). Another exploration pushed the limits of the data table to maintain a similar ordering pattern across Help Desk and Self-Service (B).

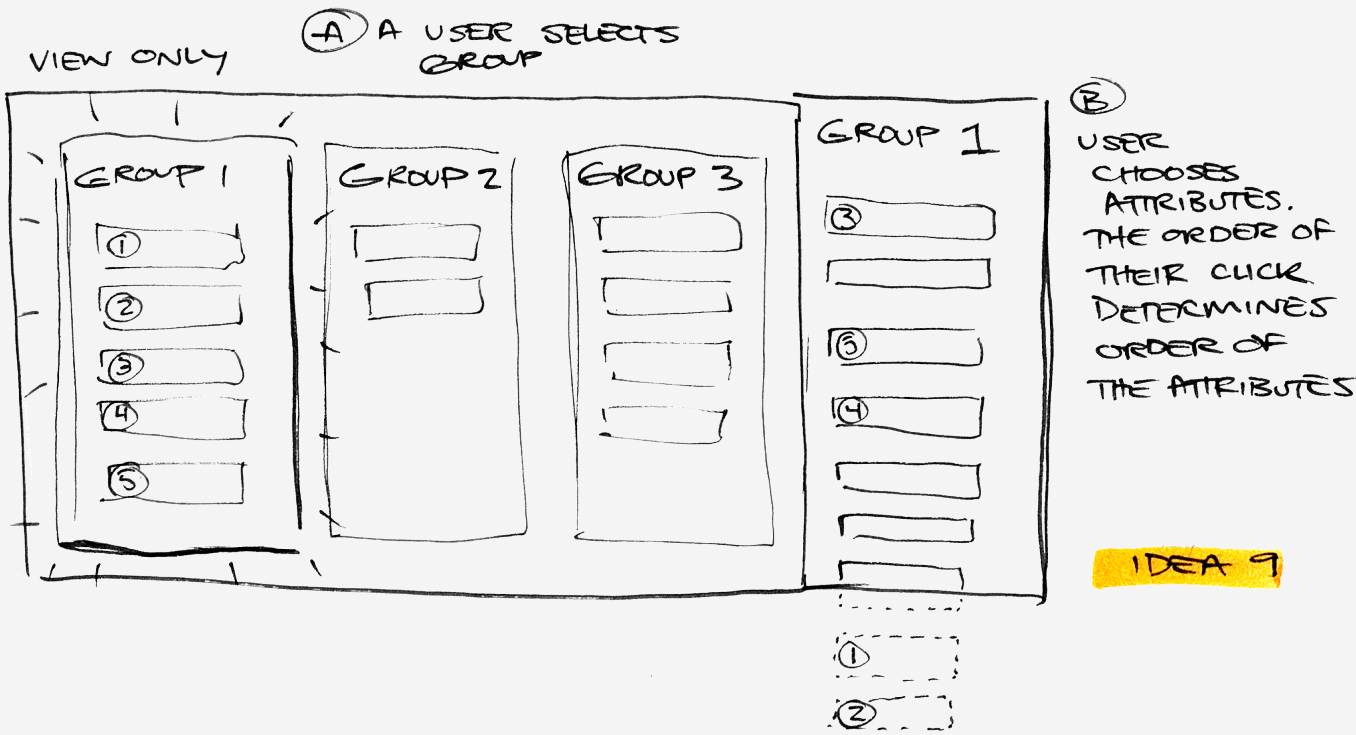
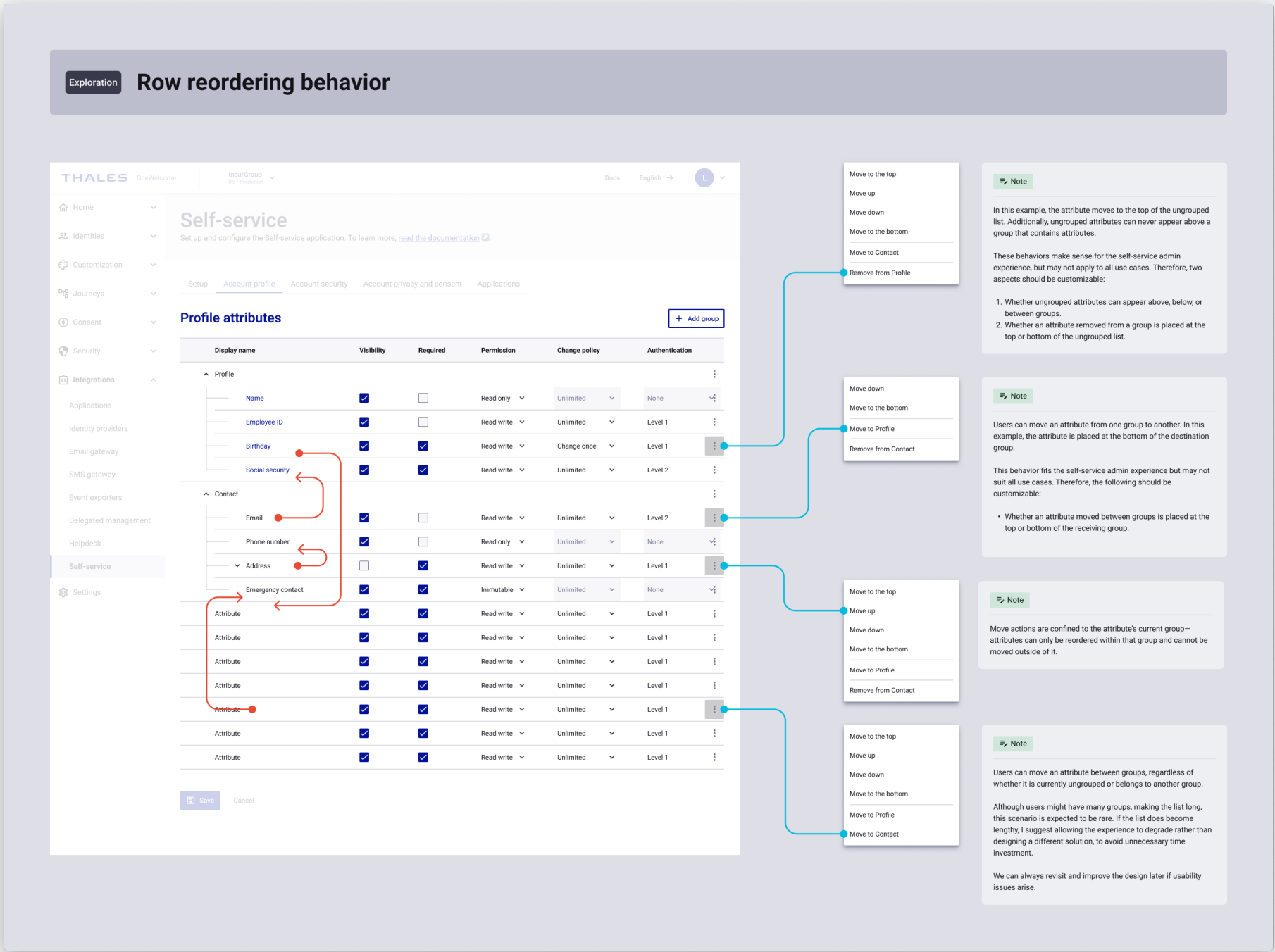
Supporting multiple pages was exclusively a Customer Identity and Access Management (CIAM) use case, and as the project progressed, we deprioritized this feature.

Ordering now became straightforward with the data table, even with SCIM 2.0's nested complex attributes, since it supported reordering and nesting.

B

Pushing data table limits

To maintain accessibility, I opted for manual reordering. Drag-and-drop was unsupported but I identified it as a future enhancement.



I designed a mutability-aware table to prevent invalid permission configurations

When admins configure attributes in the User Store, they assign a mutability value that establishes the default behavior for downstream systems (A).

Complexity arises when these mutability constraints intersect with the authorization model, the rules that decide who is allowed to do what in a system. This becomes important in Self-service and Help Desk scenarios, where end users and agents require different permissions on the same attributes.

In response, I designed the table to be mutability-aware, disabling permission options that would create invalid configurations. I then made this visible to admins by surfacing mutability logic in context, using inline explanations to clarify why certain permissions were disabled (B).

B

immutable

Q Search items

+ Select attributes

Display name	Type	Is visible	Is required	Mutability	Status
Salutation	string	☐	☐	readWrite	Active
First name	string	☐	☐	readWrite	Active
Last name	string	☐	☐	readWrite	Active
Address	complex	☐	☐	readWrite	Active
Employee ID	integer	☐	☐	immutable	Active
Job title	string	☐	☐	readOnly	Active
Department	string	☐	☐	readOnly	Active

📌 Note

When an attribute's mutability value is set to **immutable**, its visibility can be controlled, it cannot be set to required, and its mutability cannot be changed.

📌 Note

Immutable

- Can be set only once, at the time of resource creation.
- Cannot be changed or updated after that.
- The value remains readable in API responses.
- Example: Fields like `userName` or `employeeNumber` that should not change once set.

Employee ID

TH-34345

readOnly

Q Search items

+ Select attributes

Display name	Type	Is visible	Is required	Mutability	Status
Salutation	string	☐	☐	readWrite	Active
First name	string	☐	☐	readWrite	Active
Last name	string	☐	☐	readWrite	Active
Address	complex	☐	☐	readWrite	Active
Employee ID	integer	☐	☐	immutable	Active
Job title	string	☐	☐	readOnly	Active
Department	string	☐	☐	readOnly	Active

📌 Note

When an attribute's mutability value is set to **readOnly**, its visibility can be controlled, it cannot be set to required, and its mutability cannot be changed.

📌 Note

readOnly

- Visible in the UI and included in API responses.
- Cannot be modified by clients.
- Typically used for system-managed fields.
- Example: `id`, `meta.created`, or `meta.lastModified`.

Job title

Senior UX Designer

readWrite

Q Search items

+ Select attributes

Display name	Type	Is visible	Is required	Mutability	Status
Salutation	string	☐	☐	readWrite	Active
First name	string	☐	☒	readWrite	Active
Last name	string	☐	☐	Write only	Active
Address	complex	☐	☐	Read only	Active
Employee ID	integer	☐	☐	immutable	Active
Job title	string	☐	☐	readOnly	Active
Department	string	☐	☐	readOnly	Active

📌 Note

When an attribute's mutability value is set to **readWrite**, its visibility can be controlled, it can be set to required, and its mutability can be changed to **readOnly** and **writeOnly**.

📌 Note

readWrite

- Fully editable and viewable by the client.
- The most common mutability setting.
- Example: `displayName`, `email`, `phoneNumbers`.

First name

Scott

writeOnly

Q Search items

+ Select attributes

Display name	Type	Is visible	Is required	Mutability	Status
Salutation	string	☐	☐	readWrite	Active
First name	string	☐	☒	writeOnly	Active
Last name	string	☐	☐	readWrite	Active
Address	complex	☐	☐	readWrite	Active
Employee ID	integer	☐	☐	immutable	Active
Job title	string	☐	☐	readOnly	Active
Department	string	☐	☐	readOnly	Active

📌 Note

When an attribute's mutability value is set to **writeOnly**, its visibility can be controlled, it can be set to required.

📌 Note

SCIM **writeOnly** is absolute: once it's set that way, the value cannot be returned.

📌 Note

writeOnly

- Can be written or overwritten multiple times by the client.
- Not included in any API responses.
- Commonly used for sensitive data.
- Example: `password`.

First name

A

Mutability value	Attribute example	Description
immutable	Employee ID	You can only see it and it will never change.
readOnly	Job title	You can only see it.
readWrite	Name	You can see and update it at any time.
writeOnly	Password	You cannot see it, but you can update it.

Q Search items

+ Select attributes

Display name	Type	Is visible	Is required	Mutability	Status
Salutation	string	☐	☐	readWrite	Active
First name	string	☐	☒	readWrite	Active
Last name	string	☐	☐	readWrite	Active
Address	complex	☐	☐	readWrite	Active
Employee ID	integer	☐	☐	immutable	Active
Job title	string	☐	☐	readOnly	Active
Department	string	☐	☐	readOnly	Active

View First name →

To Users/Attributes/firstName

Building a poly-hierarchy

Attribute display names linked back to the User Store, allowing admins to update an attribute's mutability value when needed.

A clean attribute-to-component mapping began to break down at the edges

After defining the administrative experience, I turned my attention to how attributes would be rendered in end-user experience.

SCIM 2.0 defines multiple attribute types, and I needed to determine how each would be rendered and edited by end users (A). Our design system could support most attribute types, but headaches emerged with complex, multi-valued attributes.

A card component an appropriate choice for this attribute type, but our design system didn't have one. I partnered with a teammate to introduce a card component to the design system (B), completing predictable support for the full range of attribute types users could encounter.

Attribute type	Single-valued Component	Multi-valued Component
binary	File Uploader	File Uploaded Requires multi-select
boolean	Checkbox	n/a
complex	Form Section	Card
dateTime	Date Time Picker	Date Time Picker Requires multi-select
decimal	Number Input	Number input Requires multi-input
integer	Number Input	Number input Requires multi-input
reference	Select	Select Requires multi-select
string	Text Input	Text Input Requires multi-input

A

PROCESS

Complex multi-value attributes

Addresses are often complex multi-valued attributes and require users can enumerate more than one.

B

Addresses +

Address 1

Street address *

Primary ▾

Street address *

421 Glenrose Canyon

Apartment / Unit / Suite

Unit 210A

City *

Phoenix

State *

AZ ▾

Postal code *

85001

Address 2

Street address *

Business ▾

Street address *

1789 Willow Creek Drive

Apartment / Unit / Suite

Suite 305

City *

Denver

State *

CO ▾

Postal code *

80202

Legacy systems challenged the vision for Identifiers during implementation

When the work moved into implementation, I entered into a new set of challenges. I designed Self-Service against a platform vision where a configurable identity model enabled a composable user experience.

But technical constraints inherited from a legacy system had not been visible during earlier planning and exposed gaps between the platform's intended architecture and its current reality.

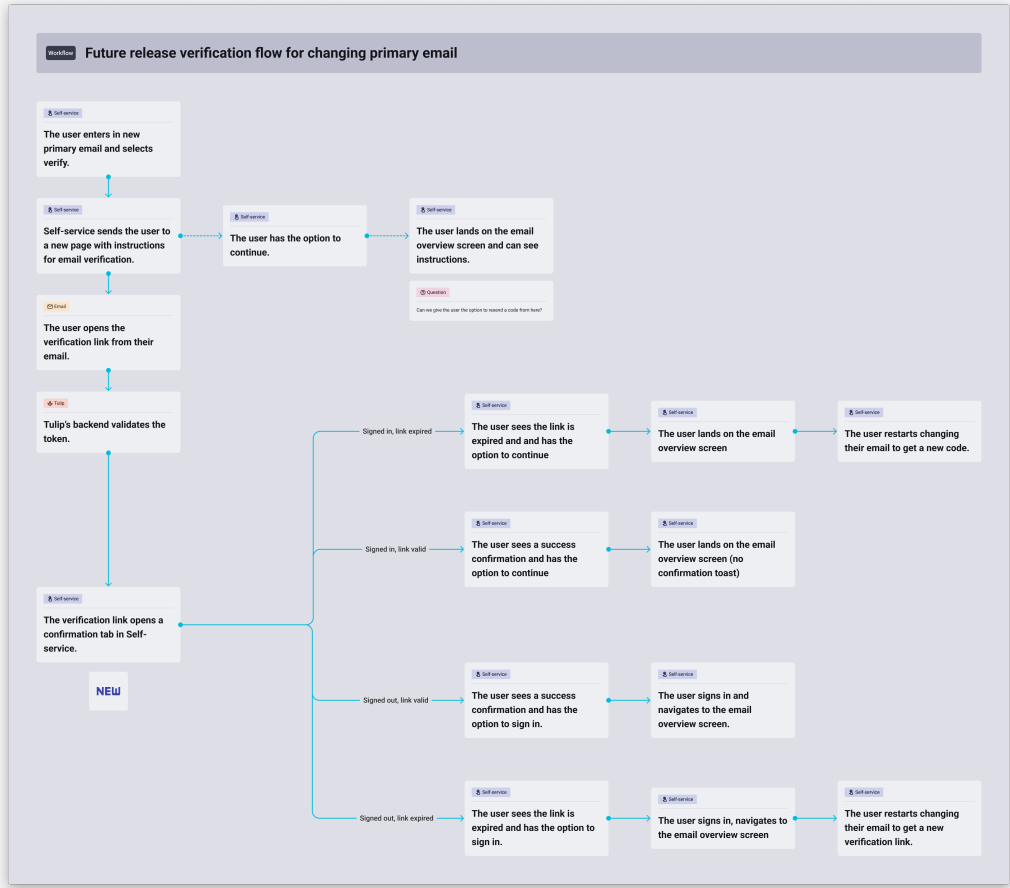
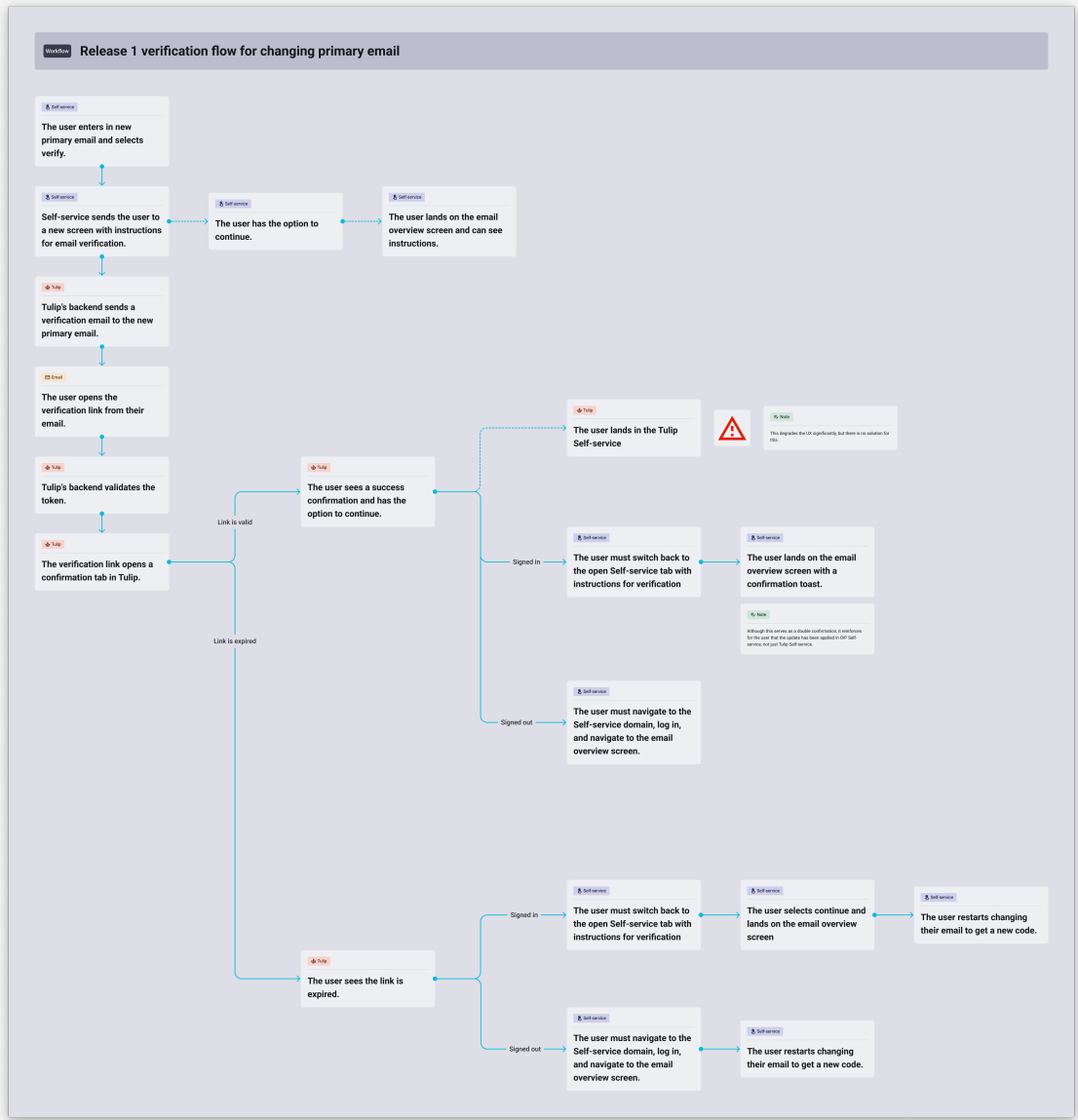
This discovery required adjusting several aspects of the Self-Service design, including how identifier attributes would be managed.

Identifiers are attributes used for sign-in, such as an email address. While the User Store allowed IAM administrators to designate any attribute as an identifier, the legacy system only supported email addresses and phone numbers.

PROCESS

Reworking verification flows

I worked with development to identify where end-user verification flows needed to be adjusted to align with the legacy system.



Paring back Identifier configuration

In the configurable design, admins defined how end-users worked with Identifiers (A), but this had to be scaled back for the first release (B).



Identifiers

Identifiers appear in a separate section of the user profile and follow their own workflow for viewing, creating, editing, and deleting.

+ Select identifier

Identifier	Is visible	Is required	Verification flow ⓘ
User name	☒	☐	Custom policy↔
Email address	☒	☐	Email Level 1↔
Phone	☒	☒	Phone Level 3↔
Employee ID	☒	☒ ⓘ	↔



Admin experience		First release

I built documentation system preserving content and decisions for other designers

When the first release of Self-Service shipped, portions of the broader vision remained unimplemented. I later shifted focus to other areas of the platform and ultimately left Thales before that work could be completed.

Knowing the work would continue without me, I took deliberate steps to ensure the design vision, rationale, and implementation context would not be lost.

Throughout the project, I documented the evolution of the experience over time by capturing design decisions, technical constraints, and metadata about each iteration (A). This enabled future designers and team members to understand not only what was designed, but also the reasoning, tradeoffs, and context behind those decisions.

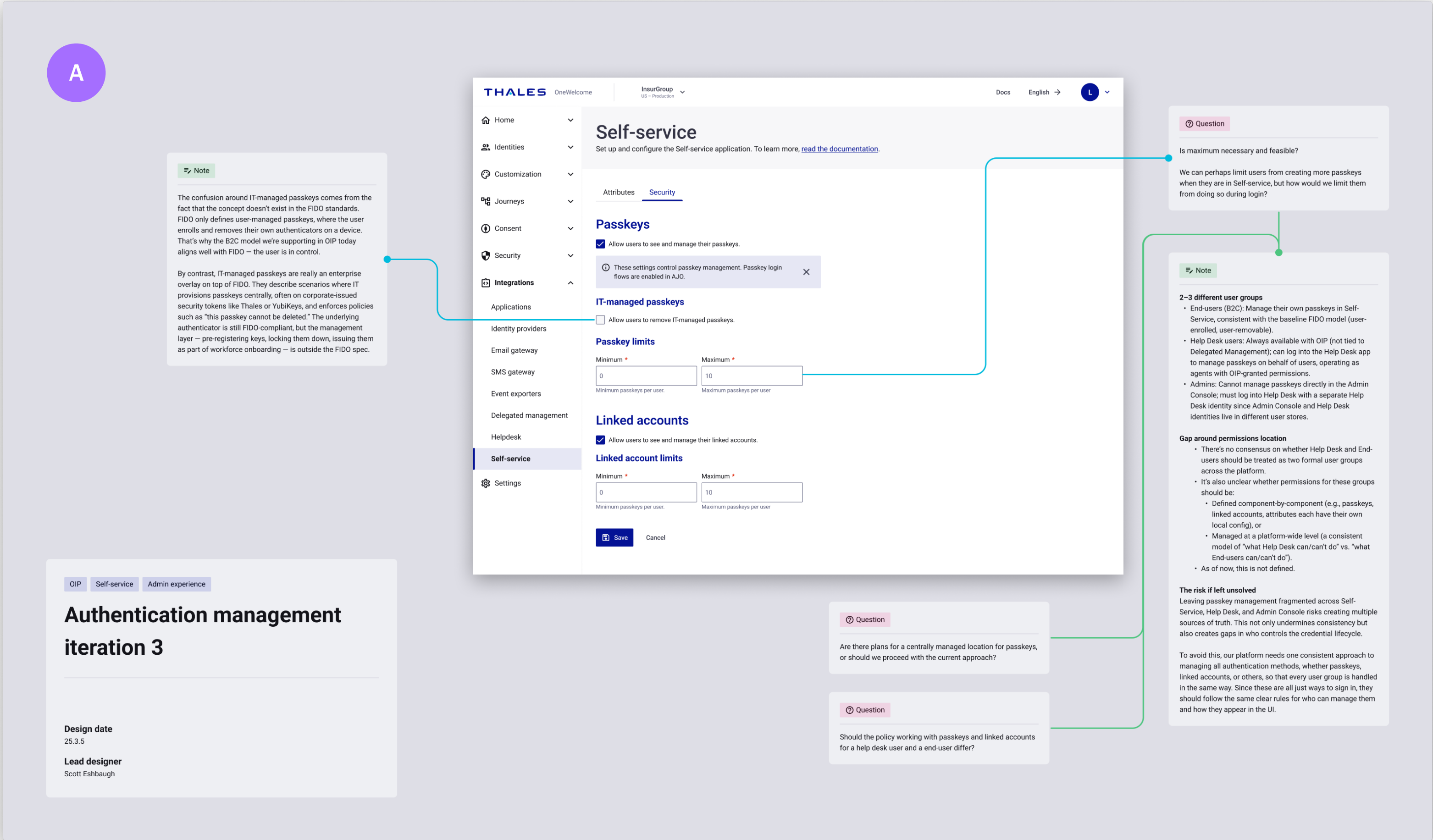
Context saves time and rework

Prior to Self-service, I introduced a documentation framework and components to elevate team standards.

PROCESS

Showing the iceberg beneath water

The passkey admin experience surfaced several platform-level gaps between product and engineering.



“In a space where scope was poorly defined across four product areas, [Scott] delivered a coherent solution through close collaboration with PM, architecture, and engineering.”

— Design Manager

Self-Service was a dormant capability with unclear requirements and dependencies

When I focused on Self-Service, it was a dormant product area with only a single feature built two years prior.

Its market positioning was conceptual, requirements were ambiguous, and platform dependencies were unclear. Self-service needed structured definition to become viable.

I drove commitment and delivered a flexible, usable self-service experience

I transformed its ambiguity into a set of decisions about what the product was becoming and how it should behave. I untangled cross-system dependencies, aligned stakeholders on a shared approach, and structured the experience around both user needs and technical constraints. As a zero-to-one effort, this work de-risked a complex initiative.

Self-Service launched shortly before my departure, so long-term user outcomes were not yet visible. Its impact is reflected in organizational and product-level outcomes: committing the organization to a self-service experience flexible enough for sophisticated configuration without sacrificing usability.

I introduced workflows and components for our platform’s growing complexity

Through designing Self-service, I introduced several improvements to our platform’s core components and interaction patterns.

At the time, the platform relied on a small set of patterns. While driving consistency, they limited efficiency for more complex use cases. I defined a new workflow for working with objects in a table, introduced inline editing, and a card component.

While initially designed for Self-service, these patterns were generalized across the platform, expanding the design system’s ability to support the growing complexity of the platform.

I extended theming, enabled full branded experiences across the platform

Last, before working on Self-service, customers could only brand login screens. I extended token-based theming across the full design system, enabling not only a customizable Self-service experience, but also unlocking branded experiences for Help Desk and other customer-facing workflows.